Author: Gabriel Vance

Class: DSC530 - Data Exporation and Analysis

Professor: Matthew Metzger

Date: April 2, 2025

Purpose: This notebook performs various tasks labeled "Exercises" for Week 3 & 4 of the class. Data must be present within the folder that this notebook is run in.

Exercise 4 - Run the code in the first cell of the exercises.ipynb notebook. It will give you a list of 100 values to work with for the rest of the exercises in this chapter. Be sure to treat these values as a sample of the population.

In [ ]:
```
import random
import pandas as pd
import numpy as np

random.seed(0)
salaries = [round(random.random()*1000000, -3) for _ in range(100)]
```

Exercise 5. Using the data from Exercise 4, calculate the following statistics without importing anything from the statistics module (Python Statistics) in the Python standard library, and then confirm your results match up to those that are obtained when using the statistics module (where possible):

In [4]:
```
salaries[:10]
# just to verify data, show the first 10
```

Out[4]:
```
[844000.0,
 758000.0,
 421000.0,
 259000.0,
 511000.0,
 405000.0,
 784000.0,
 303000.0,
 477000.0,
 583000.0]
```

In [ ]:
```python
# For my library, I chose pandas and the series object. This offered simple statist
salaries_as_series = pd.Series(salaries)
pandas_mean = salaries_as_series.mean()
pandas_median = salaries_as_series.median()
pandas_mode = salaries_as_series.mode()[0]
pandas_variance = salaries_as_series.var()
pandas_stdev = salaries_as_series.std()

pandas_results = [pandas_mean, pandas_median, pandas_mode, pandas_variance, pandas_

import statistics

mean = statistics.mean(salaries)
median = statistics.median(salaries)
mode = statistics.mode(salaries)
variance = statistics.variance(salaries)
stdev = statistics.stdev(salaries)

statistics_results = [mean, median, mode, variance, stdev]

# --- Create Dictionary for Comparison ---
keys = ["Mean", "Median", "Mode", "Variance", "Standard Deviation"]

for i in range(len(keys)):
    print(f"{keys[i]}: {pandas_results[i]} (pandas) vs {statistics_results[i]} (sta
```

```
Mean: 585690.0 (pandas) vs 585690.0 (statistics)
Median: 589000.0 (pandas) vs 589000.0 (statistics)
Mode: 477000.0 (pandas) vs 477000.0 (statistics)
Variance: 70664054444.44444 (pandas) vs 70664054444.44444 (statistics)
Standard Deviation: 265827.11382484 (pandas) vs 265827.11382484 (statistics)
```

Using the data from Exercise 4, calculate the following statistics using the functions in the statistics module where appropriate:

In [ ]:
```python
# range could be calculated with a library, but it's simple max - min
salary_range = max(salaries) - min(salaries)
# Coefficient of variation formula
# CV=( Standard Deviation / Mean)×100%

stdev = statistics.stdev(salaries)
cv = (stdev / mean) * 100
```

```
q1, q3 = np.percentile(salaries, [25, 75])
# Interquartile Range (IQR) is the difference between the 75th and 25th percentiles
iqr = q3 - q1
print(f"IQR: {iqr}")


# Coefficient of Quartile Deviation (QCD) is a measure of relative dispersion, calc
qcd = (q3 - q1) / (q3 + q1)

print(f"QCD: {qcd:.3f}")
```

```
IQR: 413250.0
QCD: 0.375
```

Exercise 7. Scale the data created in Exercise 4 using the following strategies:

In [ ]:
```
# min max scaling formula: (X - X_min) / (X_max - X_min)

X_min = np.min(salaries)
X_max = np.max(salaries)
normalized_data = (salaries - X_min) / (X_max - X_min)
normalized_data


mean = np.mean(salaries)
std_dev = np.std(salaries)

# Standardization formula: (X - mean) / std_dev
# This is also known as z-score normalization.
standardized_data = (salaries - mean) / std_dev

standardized_data
```

Out[ ]:
```
array([ 0.97661715,  0.65146878, -0.62265912, -1.23514791, -0.28238758,
       -0.68315184,  0.74976945, -1.06879293, -0.41093461, -0.01017034,
        1.21858803, -0.30507235, -1.14818962,  0.64390719,  0.12215749,
       -1.26539427,  1.22614962,  1.50214765,  0.84807012,  1.19590326,
       -1.04232737,  0.54560652,  1.18456087,  0.37168995, -0.42983858,
       -1.83251351, -0.57350879,  0.09569192,  1.237492  ,  1.44165493,
       -0.41093461,  1.05601384, -1.23136711,  0.82916615, -0.13871737,
       -2.16144268,  0.50779857, -0.70583661,  0.90478204,  0.31119723,
       -2.21059301, -0.34666109,  1.06735623, -1.29185983, -0.98561544,
        1.07491782, -1.49224197, -0.06688226, -1.31076381,  1.44543573,
        0.82160456, -0.52057766, -1.91191021, -1.00451942, -0.29372996,
        1.3131079 , -1.80226716, -0.13115578,  0.45864824, -0.14627896,
        0.8631933 , -0.17274452,  1.43031255,  0.06544556,  0.00873364,
       -0.53192004,  0.03898   , -0.75876774, -0.0366359 , -1.11794327,
       -1.49980356, -1.50736515,  0.10325351,  0.26960849, -0.41093461,
       -1.87410226,  0.65146878,  1.10138338,  1.27529995,  0.96905556,
        1.18078008,  1.27529995, -0.16896373, -0.73608297,  0.45108665,
       -1.17087439,  0.85563171,  0.99552112,  1.16943769,  0.01629523,
        1.37738142, -0.02151272, -0.50923527,  0.28095087,  1.55129798,
        1.25261518,  0.78379661, -1.90434862,  0.10325351, -0.37690745])
```

Exercise 8. Using the scaled data from Exercise 7, calculate the following:

In [ ]:
```
# Assuming normalized_data and standardized_data are NumPy arrays of the same lengt
cov_matrix = np.cov(normalized_data, standardized_data)

# Extract covariance value (off-diagonal element)
covariance = cov_matrix[0, 1]

print("Covariance Matrix:\n", cov_matrix)
print("Covariance between normalized and standardized data:", covariance)


# Assuming normalized_data and standardized_data are NumPy arrays of the same lengt
correlation_matrix = np.corrcoef(normalized_data, standardized_data)

# Extract correlation value
pearson_corr = correlation_matrix[0, 1]

print("Correlation Matrix:\n", correlation_matrix)
print("Pearson Correlation Coefficient:", pearson_corr)
```

```
Covariance Matrix:
 [[0.07137603 0.26850885]
 [0.26850885 1.01010101]]
Covariance between normalized and standardized data: 0.2685088459449036
Correlation Matrix:
 [[1. 1.]
 [1. 1.]]
Pearson Correlation Coefficient: 1.0
```

Chapter 2 Exercises 1-6

Exercise 1. Find the 95th percentile of earthquake magnitude in Japan using the mb magnitude type.

In [ ]:
```
# Read data into pandas DataFrame
df = pd.read_csv(r"C:\Users\Gabe\Documents\GitHub\data_science_classes_windows\week
japan_data = df[df['parsed_place'] == "Japan"]
```

In [ ]:
```
japan_data = df[df['parsed_place'] == "Japan"]
mb_japan_data = japan_data[japan_data['magType'] == "mb"]
percentile_95 = mb_japan_data['mag'].quantile(0.95)
percentile_95
```

Out[ ]: 4.9

Exercise 2. Find the percentage of earthquakes in Indonesia that were coupled with tsunamis.

In [ ]:
```
# Get Indonesia data
indonesia_data = df[df['parsed_place'] == "Indonesia"]
# Get the number of earthquakes with tsunami in Indonesia
indonesia_with_tsunami = df[df['tsunami'] == 1]
# Get the total number of earthquakes in Indonesia
total_earthquakes = indonesia_data.shape[0]
# Get the number of earthquakes with tsunami in Indonesia
tsunami_earthquakes = indonesia_with_tsunami.shape[0]
```

```
# Calculate the percentage of earthquakes with tsunami in Indonesia
tsunami_percentage = (tsunami_earthquakes / total_earthquakes) * 100
print(f"Percentage of earthquakes with tsunami in Indonesia: {tsunami_percentage:.2
```

```
Percentage of earthquakes with tsunami in Indonesia: 41.50%
```

Exercise 3. Calculate summary statistics for earthquakes in Nevada.

In [35]:
```
# Summary statistics means count, mean, std, min, 25%, 50%, 75%, max
# Python does this for us using the describe() method
nevada_data = df[df['parsed_place'] == "Nevada"]
nevada_data.describe()
```

Out[35]:

	cdi	dmin	felt	gap	mag	mmi	nst	
count	15.000000	681.000000	15.000000	681.000000	681.000000	1.00	681.000000	681.000
mean	2.440000	0.166199	2.400000	153.668120	0.500073	2.84	12.618209	0.151
std	0.501142	0.166228	4.626013	68.735302	0.696710	NaN	9.866963	0.084
min	2.000000	0.001000	1.000000	29.140000	-0.500000	2.84	3.000000	0.000
25%	2.000000	0.053000	1.000000	97.380000	-0.100000	2.84	6.000000	0.106
50%	2.200000	0.112000	1.000000	149.140000	0.400000	2.84	10.000000	0.146
75%	2.900000	0.233000	1.000000	199.720000	0.900000	2.84	16.000000	0.187
max	3.300000	1.414000	19.000000	355.910000	2.900000	2.84	61.000000	0.863

Exercise 4. Add a column indicating whether the earthquake happened in a country or US state that is on the Ring of Fire. Use Alaska, Antarctica (look for Antarctic), Bolivia, California, Canada, Chile, Costa Rica, Ecuador, Fiji, Guatemala, Indonesia, Japan, Kermadec Islands, Mexico (be careful not to select New Mexico), New Zealand, Peru, Philippines, Russia, Taiwan, Tonga, and Washington.

In [ ]:
```
ring_of_fire_locations = [
    "Alaska",
    "Antarctica",
    "Bolivia",
    "California",
    "Canada",
    "Chile",
    "Costa Rica",
    "Ecuador",
    "Fiji",
    "Guatemala",
    "Indonesia",
    "Japan",
    "Kermadec Islands",
    "Mexico",
    "New Zealand",
    "Peru",
```

```
    "Philippines",
    "Russia",
    "Taiwan",
    "Tonga",
    "Washington",
]
# Create a new column 'Inside_Ring_of_Fire' to indicate if the earthquake occurred
df['Inside_Ring_of_Fire'] = df['parsed_place'].apply(lambda x: 1 if x in ring_of_fi
```

Exercise 5. Calculate the number of earthquakes in the Ring of Fire locations and the number outside of them.

In [ ]:
```
# Count the number of earthquakes inside and outside the Ring of Fire
inside_ring_of_fire = df[df['Inside_Ring_of_Fire'] == 1]
outside_ring_of_fire = df[df['Inside_Ring_of_Fire'] == 0]
# Count the number of earthquakes inside and outside the Ring of Fire
print(f"Number of earthquakes inside the Ring of Fire: {inside_ring_of_fire.shape[0
print(f"Number of earthquakes outside the Ring of Fire: {outside_ring_of_fire.shape
```

```
Number of earthquakes inside the Ring of Fire: 7184
Number of earthquakes outside the Ring of Fire: 2148
```

Exercise 6. Find the tsunami count along the Ring of Fire.

In [ ]:
```
# Count the number of earthquakes with a tsunami inside the Ring of Fire
inside_ring_of_fire[inside_ring_of_fire['tsunami'] == 1].shape[0]
```

Out[ ]: 45